

Why `int_recon_mly_snapshot` Cannot Be Built in Real-Time from Kafka CDC

Architecture Decision Record — Phase 0

Service: `biq-analytics-recon-consumer` | Target Table: `interfaces.int_recon_mly_snapshot`

Background

The Kafka CDC pipeline consumes 20+ Debezium topics, one per source table. The requirement change asks these topics to populate `interfaces.int_recon_mly_snapshot` — a denormalized target table with 52 columns assembled from 15+ source tables, identified by a composite PK of `person_mbrsh_id` + `plan_cd` + `recon_month` + `cov_str_dt`.

The following questions and blockers explain why building this snapshot directly in real-time from individual CDC events is not technically feasible or operationally safe.

Q1

The team proposed inserting a partial row per CDC event with only that topic's columns populated and all others NULL — e.g. a `person_name` event inserts `first_na`, `middle_na`, `last_na` and leaves 40+ columns as NULL. Why does this not work?

ANSWER

A `person_name` change would attempt to insert:

```
first_na  = 'John'
middle_na = 'A'
last_na   = 'Smith'
plan_cd   = NULL
cov_str_dt = NULL
person_mbrsh_id = NULL <- part of PK
recon_month = NULL <- part of PK
birth_dt   = NULL
phone_num  = NULL
adr_str_1  = NULL
... all other 40+ columns = NULL
```

This approach **completely breaks the existing batch processing flow**. There are four distinct blockers:

B1

The PK Cannot Be NULL — Row Cannot Even Be Inserted

The composite PK is `person_mbrsh_id` + `plan_cd` + `recon_month` + `cov_str_dt`. The `person_name` topic carries none of these four columns. A row with NULL PKs violates the primary key constraint and **cannot be**

inserted at all. The proposal fails at the database level before any logic runs.

B2

It Conflicts Directly with the Existing Batch Processing Lifecycle

The existing flow has a well-defined 3-step lifecycle driven by three parameters:

```
As-of date      (asOfDate)    - reference date for computing recon month
Recon month     (reconMonth)   - explicit override, format YYYY-MM-DD
Carrier name    (carrierName) - optional filter for Batch 1
```

The batch runs in this strict sequence:

```
Step 1 — reconTasklet()
  UPDATE int_recon_mly_snapshot
  SET processing_status = 'O'          <- marks old records as Outdated
  WHERE processing_status = 'P'
    AND recon_month = :reconMonth
    AND carrier_name IN (:carrierName)
Step 2 — latestSnapshotPlan()
  -> calls sp_preload_latest_enrolled_plan()  <- preloads enrollment data
Step 3 — Main INSERT step
  FOR EACH enrollment record from HBX:
    1. Build dedup key:
      personMbrshId + '_' + planCd + '_' + reconMonth + '_' + covStrDt
    2. Check duplicates:
      -> In-memory ConcurrentHashMap — SKIP if key already seen
      -> DB query — SKIP if row exists with same key + status='P'
    3. Map ALL fields: first_na, last_na, birth_dt, ssn, plan_cd,
      cov_str_dt, cov_end_dt, carrier_name, address, phone, email...
    4. Set: processing_status = 'P', matched_sw = null, mul_match_sw = null
    5. INSERT complete row into int_recon_mly_snapshot
```

If the Kafka consumer inserts partial NULL rows in real-time **before** this batch runs, the following collisions occur:

Batch Step	What Breaks
Step 1 — reconTasklet	Marks all rows with processing_status='P' as 'O'. Any partial NULL rows inserted by Kafka mid-month get marked Outdated immediately, before they were ever usable.
Step 2 — Dedup check	Checks existing rows with same PK + status='P'. Kafka partial rows with NULL PKs either fail the constraint or produce phantom duplicates that confuse the dedup logic.
Step 3 — Main INSERT	Inserts a complete 52-column row for the same member. Now two rows exist — the Kafka partial row (40+ NULLs) and the batch complete row — with no clean way to merge them.

B3

The Snapshot Is Designed to Be Assembled Once Per Month, Not Incrementally

The existing flow assembles a **complete row per enrollment record in a single pass** — all 52 columns are populated together from a joined query against HBX. The **recon_month** parameter makes this an intentional monthly snapshot, not a live table. Inserting fragmented partial rows from real-time CDC events works against

this design at a fundamental level.

B4

No Single Kafka Topic Has the Full PK

Even setting aside the batch conflict, the PK resolution problem remains. None of the 20+ source tables carry all four PK columns:

Source Topic	Has person_mbrsh_id?	Has plan_cd?	Has cov_str_dt?	Has recon_m onth?
person_management.person_name	No — has person_id only	No	No	No
person_management.person_addl_attr	No — via association table	No	No	No
person_management.person_phone	No — via person_mbrsh_person_phone_ac	No	No	No
person_management.person_email	No — via person_mbrsh join	No	No	No
person_management.adr	No — via person_mbrsh_person_adr_ac	No	No	No
person_management.person_employment	No — via association	No	No	No

Every CDC event from these topics **cannot identify which snapshot row it belongs to** without resolving the PK through a chain of database joins — none of which are available in the Kafka message.

Q2

If we cannot insert with NULLs, can we instead do a live DB lookup per CDC event to resolve the PK and then UPDATE the correct snapshot row?

ANSWER

To resolve the PK for a single **person_name** CDC event, the following join chain is required:

```
SELECT
  pm.person_membership_id AS person_mbrsh_id,
  pe.enrt_cd              AS plan_cd,
  pepm.member_cov_start_dt AS cov_str_dt
FROM person_management.person_mbrsh pm
JOIN hbe.plan_enrt_person_mbrsh_ac pepm
  ON pepm.person_mbrsh_id = pm.person_membership_id
JOIN hbe.plan_enrt pe
  ON pe.plan_enrt_id = pepm.plan_enrt_id
WHERE pm.person_id = :person_id_from_cdc_event
```

This Is One Lookup Per Event Across 3 Tables — Multiplied Across 20+ Topics

- Hundreds to thousands of live DB queries per second against the source PostgreSQL database
- The source DB is a production transactional system — this level of lookup traffic is not acceptable
- Latency of each lookup adds directly to consumer lag, risking Kafka consumer group rebalancing
- Different topics have different resolution paths — each requiring its own multi-table join, not a single reusable query

Q3

There are 20+ source tables each with different join paths to the PK. How complex is the per-topic resolution, and why can't it be standardised?

ANSWER

The mapping doc defines at least 10 distinct join paths (Join 1 through Join 10) to assemble the 52 target columns. Several of these are multi-hop and cannot share a common resolution query:

- **person_addl_attr** (birth_dt, ssn, sex_cd, claim_num) — 3-hop path through person_mbrsh_person_aa_ac → person_mbrsh → PK
- **adr** (mailing address — 7 columns) — branching CASE join with 3 resolution paths depending on whether the member is a primary applicant or a dependent sharing the primary's address
- **person_phone** — join through person_mbrsh_person_phone_ac, filter by phone type = HOME_PHONE, order by primary flag and recency, LIMIT 1
- **carrier_name** — 2-hop join: plan_enrt → hpf_plan → issuer
- **payroll_office_number** — lookup into interfaces.int_reference_data by reference type PayrollOfficeNumber
- **relationship_cd** — join relationship.person_reln filtered by effv_end_dt IS NULL
- **ctry_cd** — 3-way CASE checking is_international_address_code against hbe.reference_data then looking up a 2-digit code from interfaces.int_reference_data

These Joins Involve Tables That Are Not in the Kafka CDC Stream At All

- hpf_plan, person_mbrsh_person_aa_ac, int_reference_data are purely lookup/reference tables
- They do not generate CDC events and are not present as Kafka topics
- There is no way to resolve these columns in real-time without querying the source database directly

Q4

Even if we accept some NULL columns initially, can we UPDATE each column as its topic fires and eventually reach a complete, consistent row?

ANSWER

No. Kafka topics fire independently with no coordination. This creates an unresolvable ordering problem:

- A **plan_enrt** event may arrive and create a partial row with plan_cd and cov_str_dt populated — but first_na, birth_dt, phone_num, adr_str_1 etc. are all NULL
- A **person_name** event may arrive seconds later and attempt to update first_na — but it cannot find the row because the PK (plan_cd, cov_str_dt) was not in its event

- There is **no guaranteed ordering** across topics — `person_addl_attr` may arrive before `plan_enrt`, meaning the row does not exist yet when the update fires

Result: Snapshot Is Permanently in an Inconsistent State Mid-Month

- The snapshot would contain partially populated rows at all times during the month
- Downstream Airflow jobs and matching logic depend on complete, consistent rows
- There is no event or signal that indicates a member's row is 'complete' — the consumer has no way to know when all 20+ topics have fired for the same member

Q5

Several columns need CASE logic that depends on flags from other tables (e.g. `subscr_ssn_num` depends on `is_primary_application_code` from `person_mbrsh`). Can we cache that context per member in the consumer?

ANSWER

Caching cross-table context per member in a real-time consumer is not feasible at production scale. The following columns from the mapping doc all require simultaneous data from other tables or reference data that is not in the Kafka stream:

Target Column	Requires Data From	Problem
<code>subscr_ssn_num</code>	<code>person_mbrsh.is_primary_application_code = '56'</code>	SSN is in <code>person_addl_attr</code> CDC event; flag lives in <code>person_mbrsh</code> — different topic
<code>mbr_ssn_num</code>	<code>person_mbrsh.is_primary_application_code = '57'</code>	Same cross-topic dependency
<code>is_primary</code>	<code>admin_pglogical.reference_data</code> (YesNo lookup)	Reference data not in CDC stream at all
<code>sex_cd</code>	<code>interfaces.int_reference_data</code> (Gender lookup)	<code>COALESCE(ird.reference_code, 'U')</code> — reference table not a Kafka topic
<code>ctry_cd / state_cd</code>	<code>admin_pglogical.reference_data + int_reference_data</code>	Both reference tables absent from Kafka CDC stream

These CASE expressions cannot be evaluated from a single CDC event in isolation — they require data from other tables that would need to be fetched live or cached. Neither is practical at real-time CDC throughput across 20+ topics.

Q6

`recon_month` is part of the PK. Can the real-time consumer just set it to the current month for every CDC event?

ANSWER

No. **`recon_month`** is a controlled batch parameter, not a system clock value. It cannot be hardcoded to the current month:

- The snapshot is inherently a **monthly reconciliation artifact**, not a real-time view
- The existing flow accepts `recon_month` as an explicit input parameter (format YYYY-MM-DD) that can be overridden — it is not always the calendar current month
- It also derives from **`asOfDate`**, a reference date that controls which enrollment period is reconciled — a real-time consumer has no equivalent concept
- Hardcoding current month means the snapshot is **never in a stable, queryable state** — it mutates continuously throughout the month, making it unreliable for any downstream consumer

Q7

The `processing_status` field cycles $P \rightarrow O \rightarrow I$. Why can't the real-time consumer just insert rows with `status='P'` and let the batch update it later?

ANSWER

Because the batch's **first step** — `reconTasklet` — immediately marks ALL existing 'P' rows as 'O' (Outdated) before inserting new ones. Real-time inserts mid-month would be silently wiped out before they are ever useful.

Status	Set By	Meaning
'P' — Pending	Batch INSERT step	New record ready for matching
'O' — Outdated	<code>reconTasklet</code> (Step 1)	Superseded by a newer run for same <code>recon_month</code>
'I'	Downstream matching job	Post-match status assigned by reconciliation logic

The mapping doc specifies `processing_status` is populated as 'P' on insert, then updated to 'O' or 'I' by downstream matching logic between the snapshot and `int_recon_menr_in_mly_stg`. Similarly, `matched_sw`, `mapped_sw`, `mul_match_sw`, and comments are all initially NULL and populated by a subsequent matching process.

This is a **multi-phase processing pipeline**, not a single-pass real-time write. A real-time CDC consumer writing partial rows mid-month would interfere with or corrupt this lifecycle — every partial row inserted with `status='P'` is guaranteed to be marked Outdated the next time `reconTasklet` runs.

Q8

Even if real-time CDC is not viable, can the Airflow DAG read Bronze incrementally using `consumer_ts` instead of doing a full join every run?

ANSWER

No. Incremental processing on Bronze fails for the same structural reason as real-time CDC. Consider this typical arrival sequence across topics:

```
consumer_ts 10:00:01 -> bronze.person_name      { person_id, first_name, last_name }
consumer_ts 10:00:02 -> bronze.person_addl_attr { person_additional_attribute_id, birth_dt
, ssn }
consumer_ts 10:00:03 -> bronze.plan_enrt        { plan_enrt_id, enrt_cd, cov_str_dt } <-
PK columns
```

The Incremental Bronze Problem

If the incremental DAG queries **WHERE `consumer_ts` > `last_run_ts`**:

- It picks up `person_name` and `person_addl_attr` changes in this window
- But those Bronze rows **do not contain** `plan_cd` or `cov_str_dt`
- To resolve the snapshot PK, it must join Bronze `person_name` → Bronze `person_mbrsh` → Bronze `plan_enrt`
- But `plan_enrt` was **not updated in this window** — it sits outside the `consumer_ts` filter

- The join either returns **no rows** (if strictly filtered) or requires reading the **full Bronze table anyway** — defeating the purpose of incremental

There Is No Safe Incremental Boundary

Unlike a single-table ETL where incremental on `updated_at` is straightforward, this snapshot requires **20+ tables all consistent at the same point in time**. There is no `consumer_ts` watermark that guarantees all tables contributing to a given member's row have been captured in the same window. A name change and a plan enrollment change for the same member could be hours or days apart across different topics, making any incremental slice inherently incomplete.

The Existing Flow Confirms This — It Always Does a Full Read

The existing batch flow already proves that full refresh is the only correct approach. It runs **`sp_preload_latest_enrolled_plan()`** which performs a **complete join across all source tables** for the given `recon_month` and `carrier_name` — every run, with no incremental optimization. This is not an oversight; it is the only way to guarantee a consistent, complete snapshot for a monthly reconciliation period.

Recommended Architecture

```

Kafka Topics (20+ tables, continuous)
  |
  | existing consumer — no changes needed
  v
Iceberg Bronze (S3, partitioned by run_date)
  bronze.person_name | bronze.plan_enrt | bronze.person_mbrsh
  bronze.person_addl_attr | bronze.adr | bronze.person_phone | ...
  |
  | Airflow DAG — runs monthly, FULL refresh per recon_month + carrier_name
  | mirrors existing sp_preload_latest_enrolled_plan() logic
  v
interfaces.int_recon_mly_snapshot
-> Step 1: reconTasklet — mark existing 'P' rows -> 'O'
-> Step 2: full JOIN across all Bronze tables for the recon period
-> Step 3: INSERT complete rows with all 52 columns populated
        processing_status = 'P'

```

Why Full Refresh Is the Only Correct Approach

Question	Answer
Can we do incremental using consumer_ts?	No — non-PK tables arrive without PK context; PK tables may not be in the same window
Can we do incremental using run_date partition?	No — a name change on day 5 and a plan change on day 12 are in different partitions
Can we join only the changed Bronze tables?	No — the full PK resolution chain always requires joining across all tables simultaneously
Does the existing flow do incremental?	No — sp_preload_latest_enrolled_plan() always reads the full enrolled population for the recon period

What This Means in Practice

- The Kafka consumer and Bronze layer remain **exactly as built today** — no changes
- The Airflow DAG reads the **latest state** of each Bronze table for the recon month (most recent record per natural key using run_date partitioning)
- The full join SQL mirrors the mapping doc — all 52 columns assembled in one pass
- The snapshot is always **complete and consistent** at the time it is built, matching the cadence of the existing flow
- recon_month, asOfDate, and carrierName are passed as DAG parameters exactly as they are today

Summary — All Blockers at a Glance

#	Question / Blocker	Impact if Ignored
Q 1	Insert-NULLs proposal breaks existing batch flow	PK constraint violation; reconTasklet wipes rows; dedup creates phantoms; duplicate complete rows
Q 2	PK resolution needs live DB queries per event	Source DB overload; consumer lag; Kafka rebalancing
Q 3	20+ tables with different join depths	Each event needs a unique multi-hop resolution — not scalable per-event
Q 4	Out-of-order partial rows across topics	Snapshot permanently in inconsistent state mid-month; no signal row is 'complete'
Q 5	CASE logic needs cross-table context simultaneously	subscr_ssn_num, sex_cd, is_primary, ctry_cd cannot be evaluated from a single event
Q 6	recon_month is a monthly parameter by design	Cannot be hardcoded to current month; snapshot never reaches a stable state
Q 7	processing_status lifecycle is P → O → I	reconTasklet immediately marks real-time inserts as Outdated; lifecycle corrupted
Q 8	Incremental Bronze processing also fails	Non-PK tables land outside the PK table's consumer_ts window; full read always required

Conclusion: The existing Kafka → Bronze architecture is correct and complete. The snapshot must be assembled by a scheduled Airflow DAG using a full join across Bronze tables, mirroring the logic of the existing `sp_preload_latest_enrolled_plan()` flow. No real-time or incremental approach is architecturally viable given the composite PK structure, the multi-table join requirements, and the monthly reconciliation lifecycle of the target table.